7강. 컴포넌트 스타일링

React

리엑트 CSS

- **index.css**

  프로젝트 전체에 적용되는 기본 스타일 특징

  - 모든 컴포넌트에 영향을 줌
  - HTML 기본 스타일 초기화(reset) 또는 공통 스타일 정의
  - 폰트, 배경, 기본 색상 설정

```
/* index.css */
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #f5f5f5;
}

h1 {
  color: #333;
}
```

리엑트 CSS

- **App.css**

특징

- 특정 컴포넌트에만 적용
- 보통 `App.jsx` UI 스타일 담당

```
/* App.css */
.container {
  text-align: center;
  padding: 20px;
}

.button {
  background-color: blue;
  color: white;
}
```

리엑트 CSS

- **index.css vs App.css**

구분	index.css	App.css
범위	전역(Global)	컴포넌트(Local 느낌)
적용 대상	전체 앱	특정 컴포넌트
import 위치	main.jsx	App.jsx
용도	초기화, 공통 스타일	UI 스타일

컴퓨터 기기 쇼핑몰

컴퓨터 기기 쇼핑몰

Home 상품 목록 상품 등록 로그인

상품 리스트

상품ID	상품명	가격	설명
1	모니터	130000	22인치 최신 모니터입니다.
2	마우스	30000	무선 마우스입니다.
3	키보드	50000	무선 키보드입니다.

상품 등록하기

컴퓨터 기기 쇼핑몰

Home 상품 목록 상품 등록 로그인

상품 정보

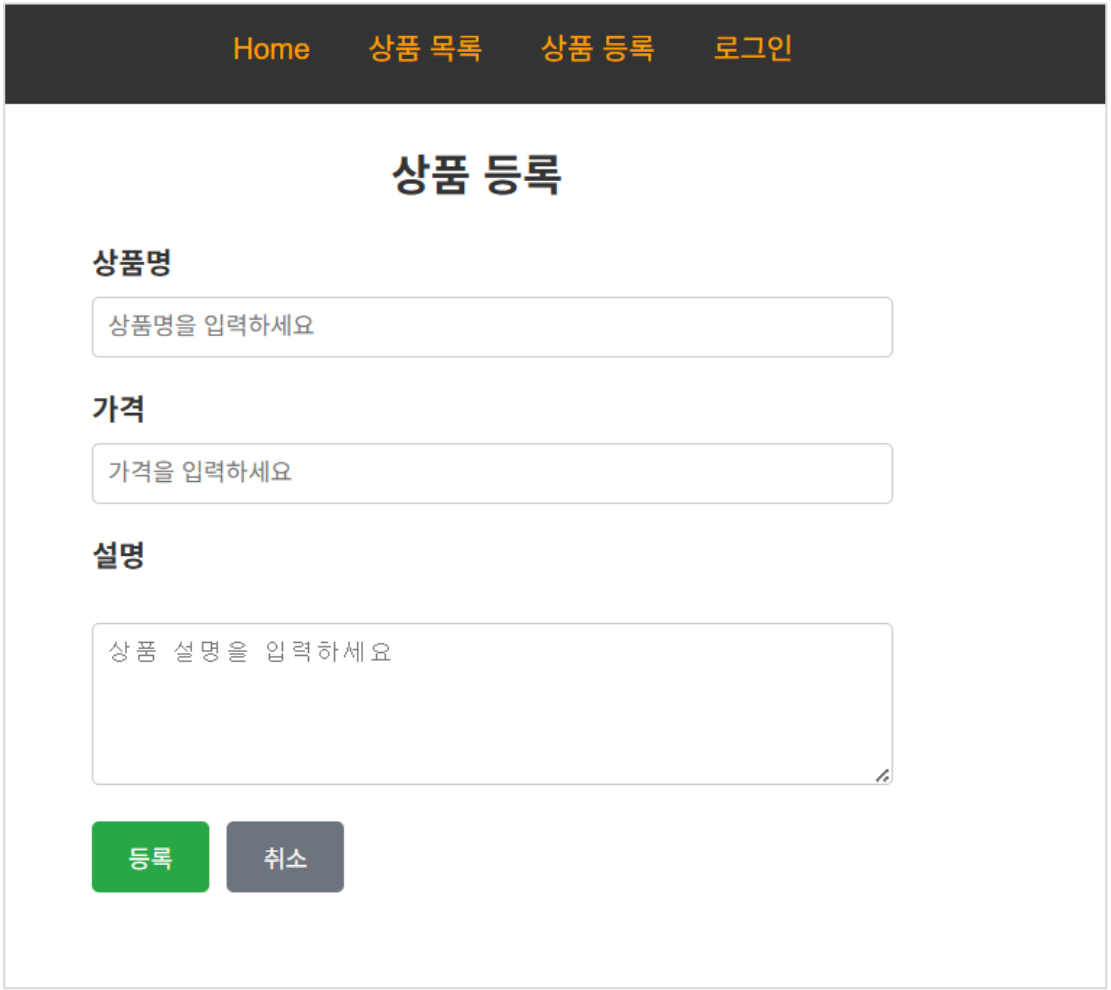
상품 ID: 2

상품 이름: 마우스

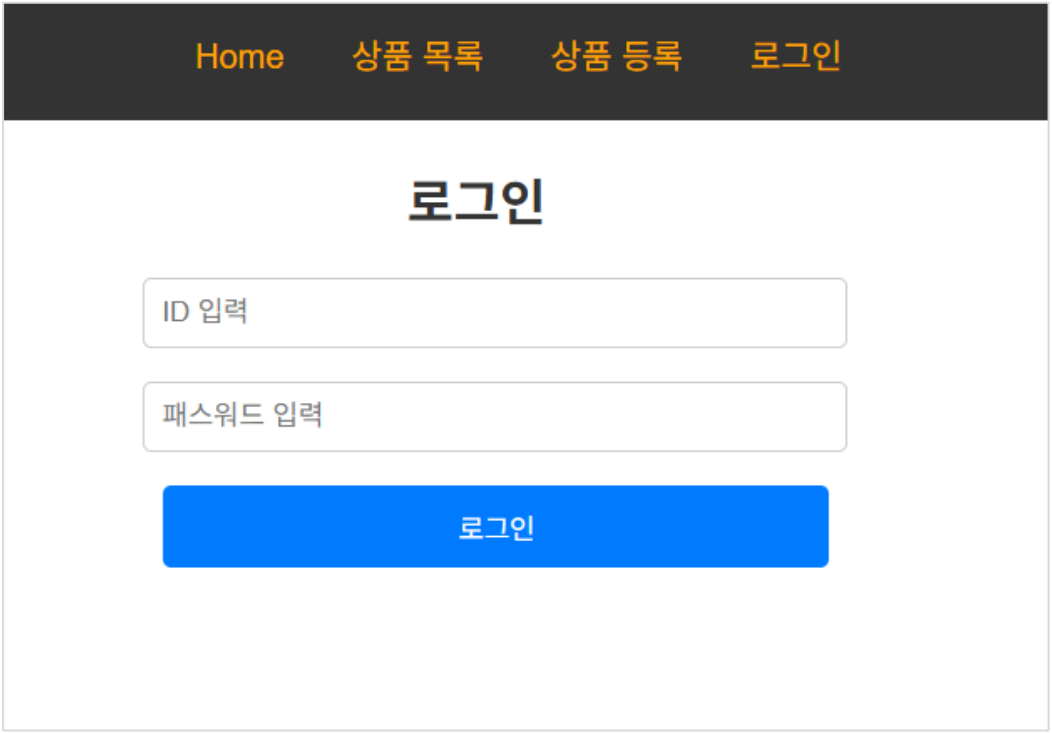
상품 가격: 30000

상품 설명: 무선 마우스입니다.

컴퓨터 기기 쇼핑몰


Home
상품 목록
상품 등록
로그인
상품 등록
상품명
상품명을 입력하세요
가격
가격을 입력하세요
설명
상품 설명을 입력하세요
등록
취소

컴퓨터 기기 쇼핑몰



컴퓨터 기기 쇼핑몰

- index.css

```
*{
  margin: 0;
  padding: 0;
}
body {
  font-family: Arial, sans-serif;
  color: #333;
}
a {
  color: #333;
  text-decoration: none;
}
a:hover {
  text-decoration: underline;
}
h1, h2, h3 {
  margin: 20px;
  text-align: center;
}
```

컴퓨터 기기 쇼핑몰

- **App.css**

```
/* Header 스타일 */
.header{
  width: 100%;
  background-color: ■#333;
  padding: 20px;
  text-align: center;
}
.header a {margin: 0 16px; color: orange; text-decoration: none;}
.header a:hover {text-decoration: underline;}
.header-user {
  display: inline-flex;
  align-items: center;
  gap: 10px;
  color: □#fff;
  margin-left: 12px;
}
```

컴퓨터 기기 쇼핑몰

- **App.css**

```
.logout-btn {
  padding: 6px 12px;
  border: 1px solid #ff9f1c;
  border-radius: 6px;
  background-color: transparent;
  color: #ff9f1c;

}
.logout-btn:hover {background-color: #ff9f1c;color: #333;}

/* Main 스타일 */
.main {
  width: 1000px;
  margin: 0 auto;
  padding: 20px;
  text-align: center;
}
.main img {margin-top: 20px;}
```

컴퓨터 기기 쇼핑몰

- App.css

```
/* ProductList 스타일 */
.product-list {
  width: 1000px;
  margin: 0 auto;
  text-align: left;
}
.table-list {
  width: 60%;
  margin: 0 auto;
  border-collapse: collapse;
}
.table-list th, .table-list td {
  border: 1px solid #ddd;
  padding: 8px;
}
.table-list th {text-align: center; background-color: #f2f2f2;}
```

컴퓨터 기기 쇼핑몰

- App.css

```
.table-list th {text-align: center; background-color: #f2f2f2;}
.btn-add {
  text-align: right;
  margin-top: 20px;
  margin-right: 20%;
}
.btn-add button {
  padding: 10px 20px;
  border-radius: 4px;
  background-color: #007bff;
  color: white;
  border: none;
  cursor: pointer;
}
.btn-add button:hover {background-color: #0056b3;}
```

컴퓨터 기기 쇼핑몰

- App.css

```
/* 상품 정보 */
.product-details {
  width: 50%;
  margin: 0 auto;
  text-align: left;
  border: 1px solid #ddd;
  padding: 20px;
  border-radius: 8px;
  background-color: #f0f0f0;
}
.product-details p {margin-bottom: 10px;}

/* 상품 등록 폼 */
.add-form {width: 40%; margin: 0 auto; text-align: left;}
.add-form label {
  display: block;
  margin-bottom: 8px;
  font-weight: bold;
}
```

컴퓨터 기기 쇼핑몰

- **App.css**

```
.add-form input,
.add-form textarea {
  width: 100%;
  padding: 8px;
  margin-bottom: 16px;
  border: 1px solid #ccc;
  border-radius: 4px;
}
.add-form button {
  padding: 10px 20px;
  border-radius: 4px;
  background-color: #28a745;
  color: white;
  border: none;
  cursor: pointer;
}
.add-form button:hover {background-color: #218838;}
.add-form button[type="submit"] {margin-right: 10px;}
.add-form button[type="button"] {background-color: #6c757d}
.add-form button[type="button"]:hover {background-color: #5a6268;}
```

컴퓨터 기기 쇼핑몰

- **App.css**

```
/* 로그인 폼 */
.signin-form {width: 30%; margin: 0 auto; text-align: left;}
.signin-form input {
  width: 100%;
  padding: 8px;
  margin-bottom: 16px;
  border: 1px solid #ccc;
  border-radius: 4px;
}
.signin-form button {
  width: 100%;
  padding: 10px 20px;
  border-radius: 4px;
  background-color: #007bff;
  color: white;
  border: none;
  cursor: pointer;
  margin-left: 3%;
}
.signin-form button:hover {background-color: #0056b3;}
```

미디어 쿼리

- **Media Query**
  - 접속하는 장치(미디어)에 따라 **특정한 CSS 스타일을 사용**하도록 함
  - 반응형 웹디자인에서 가장 많이 사용하는 방식
    (반응형 레이아웃 - 가변그리드 레이아웃, 플렉스박스 레이아웃 방식 등)
  - 미디어 쿼리 구문

@media 미디어 유형 [and 조건] * [and 조건]

예) 미디어 유형이 'screen'이면서 최소 너비가 '600px'이고 최대 너비가 '1400px' 정의

```
<style>
    @media screen and (min-width: 768px) and (max-width: 1400px){
        ....
    }
</style>
```

미디어 쿼리

- **Media Query**

```
<title>미디어 쿼리 예제</title>
<style>
  @media screen and (max-width: 767px) {
    body{
      background-color: lightblue;
    }
  }

  @media screen and (min-width: 768px) {
    body{
      background-color: lightyellow;
    }
  }

  @media screen and (min-width: 1200px) {
    body{
      background-color: lightgreen;
    }
  }
```

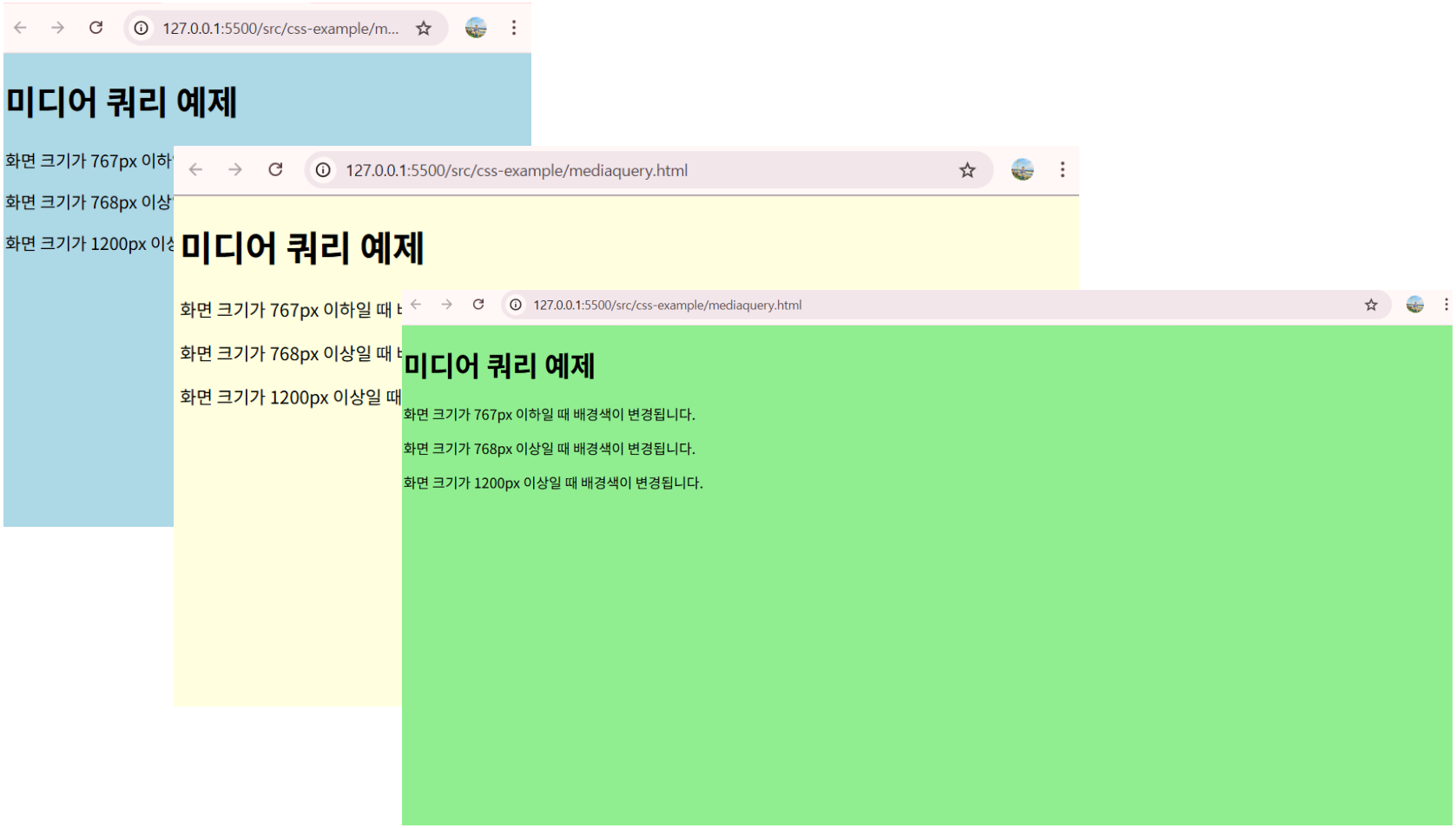
미디어 쿼리

- **Media Query**

```
<body>
  <h1>미디어 쿼리 예제</h1>
  <p>화면 크기가 767px 이하일 때 배경색이 변경됩니다.</p>
  <p>화면 크기가 768px 이상일 때 배경색이 변경됩니다.</p>
  <p>화면 크기가 1200px 이상일 때 배경색이 변경됩니다.</p>
</body>
```

미디어 쿼리

- **Media Query**



styled-components

- **리엑트 styled-components 란?**

  CSS를 JavaScript 안에서 작성할 수 있게 해주는 대표적인 **CSS-in-JS 라이브러리**입니다.

  즉, 기존처럼 .css 파일을 따로 만드는 대신

  JS 파일 안에서 스타일 + 컴포넌트를 함께 작성합니다.

- **styled-components 설치**

```
npm install styled-components
```

```
"dependencies": {
  "axios": "^1.14.0",
  "react": "^19.2.4",
  "react-dom": "^19.2.4",
  "styled-components": "^6.3.12"
},
```

styled-components

- 버튼 컴포넌트 스타일링

기본 버튼 | 녹색 버튼 | 보라 버튼

```
▼<div id="root">
  ▼<div class="App">
      <button class="sc-bpqTzb ejYZbX">기본 버튼</button> == $0
      <button color="green" class="sc-bpqTzb doGfUH" style="background-color:
      low;">녹색 버튼</button>
      <button class="sc-bpqTzb kvefPX">보라 버튼</button>
    </div>
  </div>
  <script type="module" src="/src/main.jsx?t=1774847964199"></script>
```

styled-components

- 버튼 컴포넌트 스타일링

```
const ButtonSample = () => {
  return (
    <>
      <Button>기본 버튼</Button>
      {/* props를 이용하여 스타일을 동적으로 변경 */}
      <Button color="green" backgroundColor="yellow">
        녹색 버튼
      </Button>
      {/* primary prop이 true일 때 추가 스타일 적용 */}
      <Button primary>보라 버튼</Button>
    </>
  );
};

export default ButtonSample;
```

styled-components

- 버튼 컴포넌트 스타일링

```
import styled from 'styled-components';

const Button = styled.button`
  margin: 5px;
  padding: 8px 16px;
  border: 1px solid #ccc;
  border-radius: 8px;
  font-size: 1rem;
  line-height: 1.5; // 버튼의 높이를 텍스트의 높이에 맞게 조정

  // props를 이용하여 스타일을 동적으로 변경
  color: ${props => props.color || 'black'};
  background-color: ${props => props.backgroundColor || 'white'};

  // primary prop이 true일 때 추가 스타일 적용
  ${props => props.primary && `
    color: white;
    background-color: purple;
  `}
`;
```

styled-components

- **동적 스타일링 문법**

  `${props => props.color}`는 styled-components의 동적 스타일링 문법입니다.

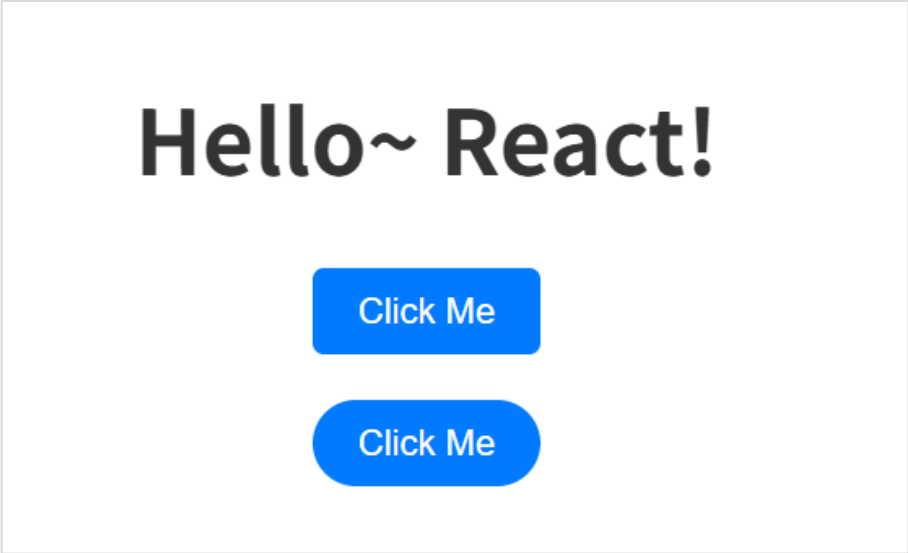
  1.`${}` = JavaScript 템플릿 리터럴 (백틱 내에서 JS 실행)

  2.`props => props.color` = 화살표 함수로 props 객체에서 color 값 추출

  3.결과 = 컴포넌트에 전달된 `color` prop을 CSS 값으로 사용

styled-components

- **Main 페이지 스타일링**



styled-components

- **Main 페이지 스타일링**

```
const MainPage = () => {
  return (
    <Wrapper>
      <Title>Hello~ React!</Title>
      <Button>Click Me</Button>
      <RoundedButton>Click Me</RoundedButton>
    </Wrapper>
  );
}

export default MainPage;
```

styled-components

- **Main 페이지 스타일링**

```
import styled from 'styled-components';

const Wrapper = styled.div`
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  // 화면 전체 높이(viewport height) : 100 * 8 = 800px
  height: 100vh;
`;

const Title = styled.h1`
  font-size: 2.5rem;
  color: #333;
  margin-bottom: 20px;
`;
```

styled-components

- 컴포넌트 확장(상속)

```
const Button = styled.button`
  padding: 10px 20px;
  margin: 10px;
  font-size: 1rem;
  color: #fff;
  background-color: #007bff;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  &:hover {
    background-color: #0056b3;
  }
`;

/* RoundedButton 컴포넌트는 Button 컴포넌트를
   상속받아 border-radius를 50px로 설정*/
const RoundedButton = styled(Button)`
  border-radius: 50px;
`;
```

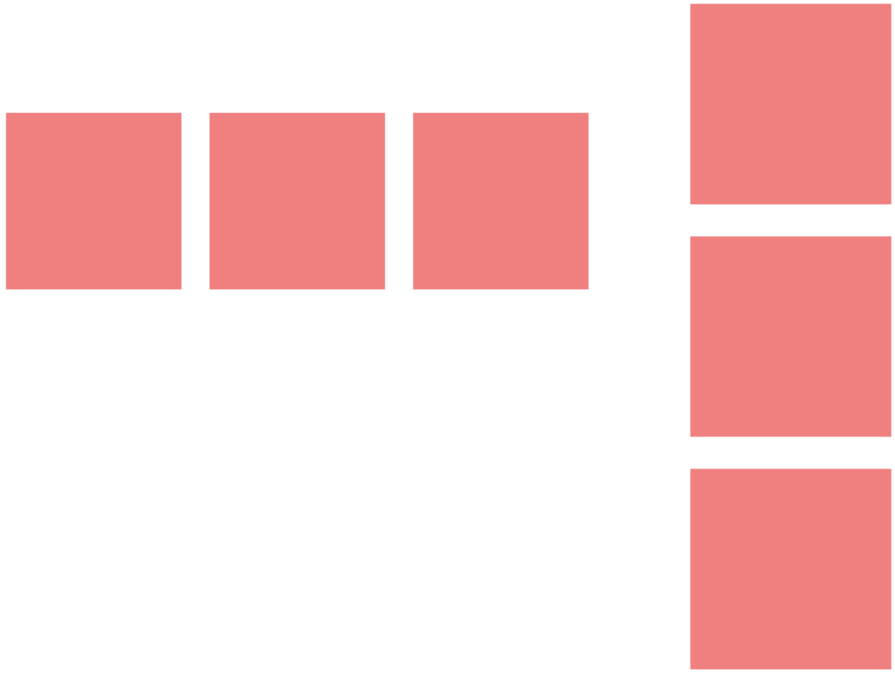
레이아웃

- **플렉스 박스(Flex Box) 레이아웃**
  - 수평이나 수직 방향 중 하나를 주축으로 정하고 박스를 배치
  - 여유 공간이 생길 경우 너비나 높이를 적절하게 늘리거나 줄일 수 있음

flex container

레이아웃 – flex

- flex-direction : 가로(row), 세로(column) 방향 배치



레이아웃 – flex

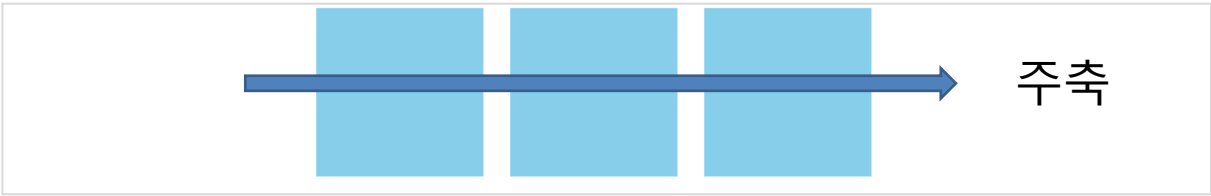
- **flex-direction : 가로(row), 세로(column) 방향 배치**

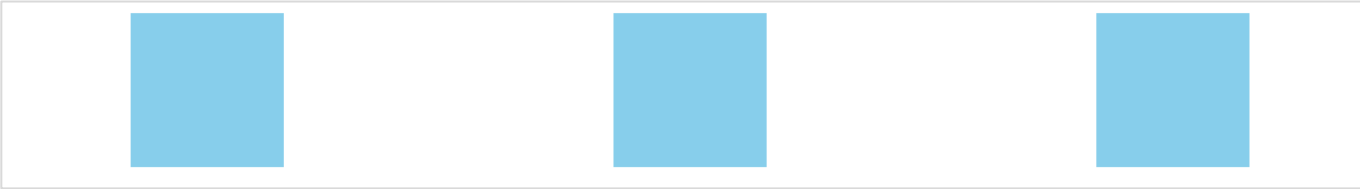
```style
<style>
  .container {
    display: flex;
    flex-direction: row; /* 아이템을 가로 방향으로 배치 */
    flex-direction: column; /* 아이템을 세로 방향으로 배치 */
    gap: 16px; /* 아이템 사이의 간격 */
  }
  .box {
    width: 100px;
    height: 100px;
    background-color: lightcoral;
  }
</style>
```

```div
<div class="container">
  <div class="box"></div>
  <div class="box"></div>
  <div class="box"></div>
</div>
```

레이아웃 – flex

- justify-content : 주축(main axis)의 정렬 방법을 지정





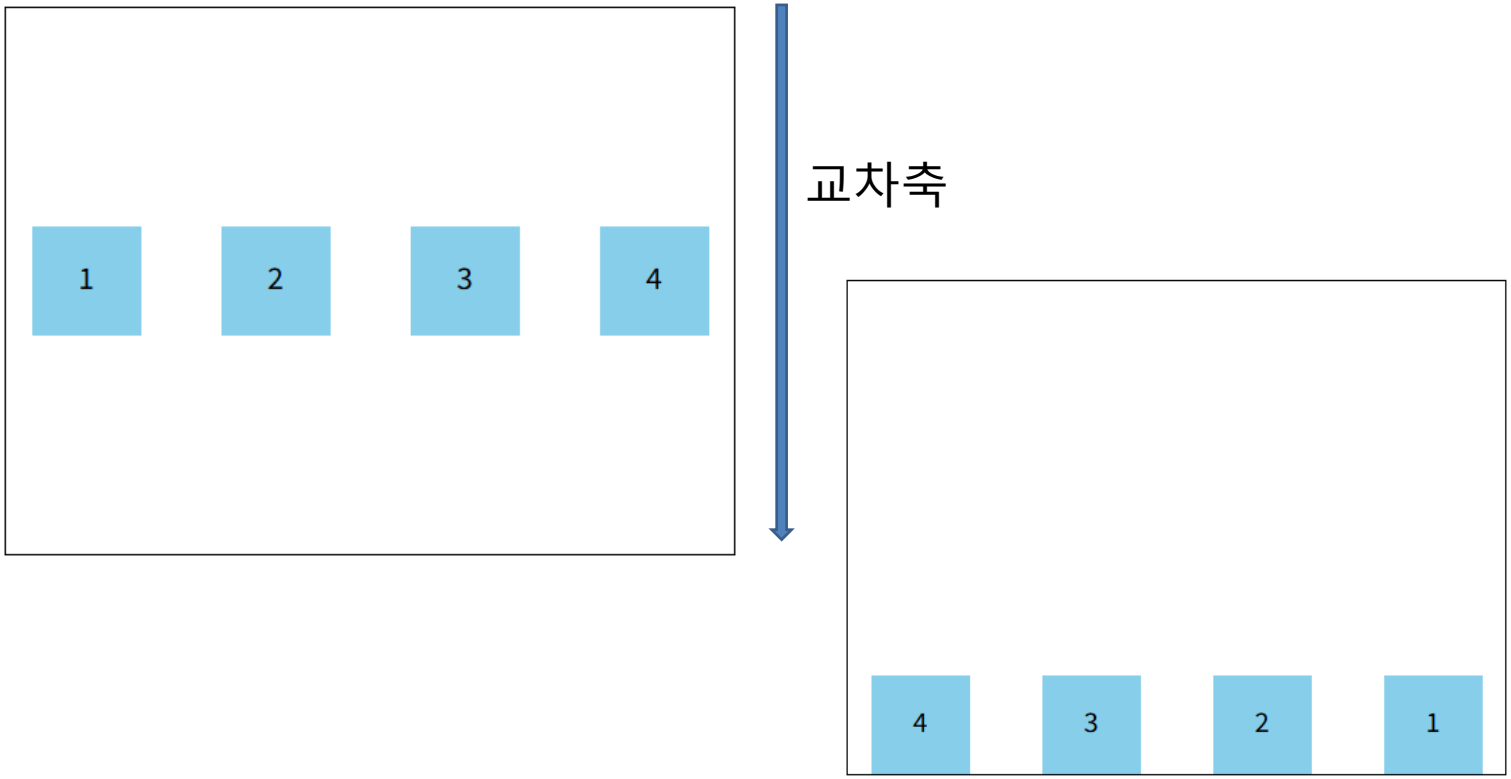
레이아웃 – flex

- **justify-content : 주축(main-axis)의 정렬 방법을 지정**

```
<style>
  .container {
    display: flex;
    justify-content: center;  /* 아이템을 가로 방향으로 중앙 정렬 */
    justify-content: flex-start;  /* 아이템을 가로 방향으로 시작 위치에 정렬 */
    justify-content: space-between;  /* 아이템을 가로 방향으로 양 끝에 정렬 */
    justify-content: space-around; /* 아이템을 가로 방향으로 고르게 분배 */
    gap: 16px; /* 아이템 사이의 간격 */
  }
  .box {
    width: 100px;
    height: 100px;
    background-color: skyblue;
  }
</style>
```

레이아웃 – flex

- **align-items : 교차축(cross axis)의 정렬 방법을 지정**



레이아웃 – flex

- **align-items : 교차축(cross-axis)의 정렬 방법을 지정**

```css
.container {
  width: 400px;
  height: 300px;
  border: 1px solid #000;
  display: flex;
  align-items: center; /* 아이템을 세로 방향으로 중앙 정렬 */
  align-items: start; /* 아이템을 세로 방향으로 시작 위치에 정렬 */
  align-items: end; /* 아이템을 세로 방향으로 끝 위치에 정렬 */
  justify-content: space-around; /* 아이템을 가로 방향으로 고르게 분배 */
  justify-content: flex-start;  /* 아이템을 가로 방향으로 시작 위치에 정렬 */
  flex-direction: row-reverse; /* 아이템을 가로 방향으로 역순 배치 */
  gap: 16px; /* 아이템 사이의 간격 */
}
.box {
  width: 60px;
  height: 60px;
  background-color: skyblue;
}
p{text-align: center;}
```

```html
<div class="container">
  <div class="box"><p>1</p></div>
  <div class="box"><p>2</p></div>
  <div class="box"><p>3</p></div>
  <div class="box"><p>4</p></div>
</div>
```

뷰포트(Viewport) 단위

뷰포트(viewport)

- 실제 내용이 표시되는 영역
- PC화면과 모바일 화면의 픽셀 표시 방법이 다르기 때문에 모바일 화면에서 의도한 대로 표시되지 않음
- 뷰포트를 지정하면 기기 화면에 맞춰 확대/축소해서 내용 표시

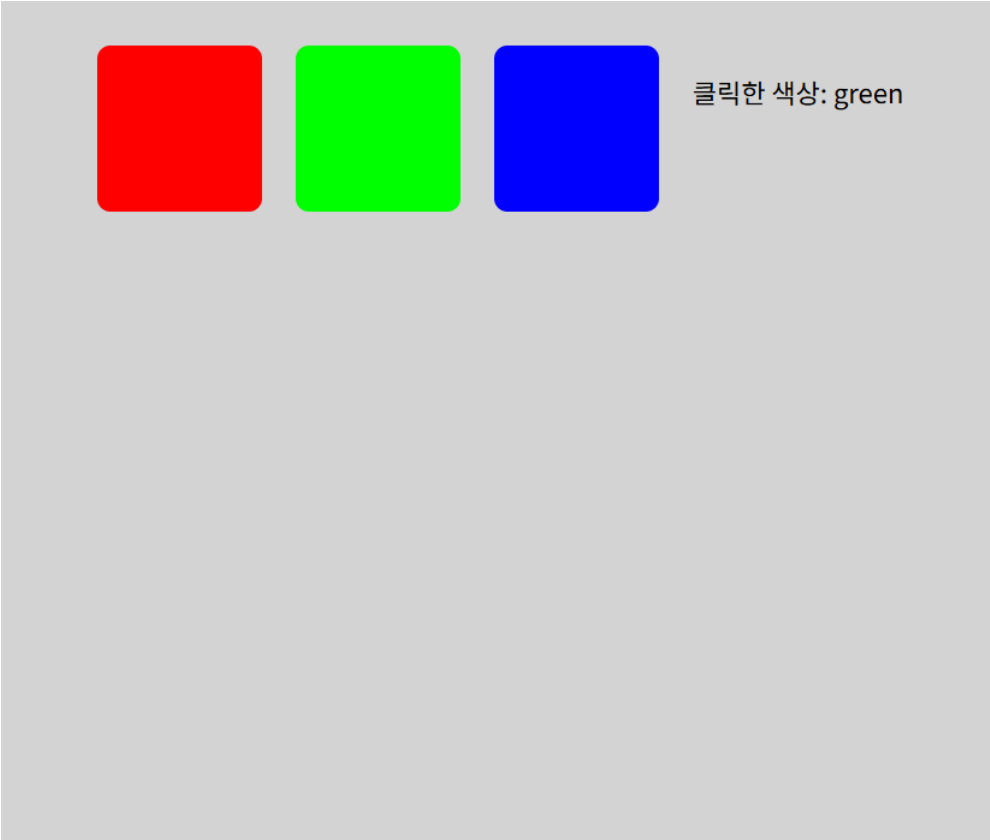
뷰포트의 단위

CSS에서 크기를 지정할 때 주로 px, % 단위는 고정 폰트이고, 브라우저의 비율에 따라 글자 크기가 변경되는 가변 폰트는 vw, vh이다.

- vw(viewport width) : 1vw는 뷰포트 너비의 1%와 같다.(1200px일때 5%-60px)
- vh(viewport height): 1vh는 뷰포트 높이의 1%와 같다.

styled-components

- **Blocks 모형**



styled-components

- **Blocks 모형**

```
const Blocks = () => {
  // 클릭한 블록의 색상을 저장하는 상태
  const [clicked, setClicked] = useState(null);

  const handleClick = (color) => {
    setClicked(color);
  };

  return (
    <Wrapper>
      <Block
        color="#f00"
        onClick={() => handleClick('red')}
      />
```

styled-components

- **Blocks 모형**

```
        <Block
          color="#0f0"
          onClick={() => handleClick('green')}
        />
        <Block
          color="#00f"
          onClick={() => handleClick('blue')}
        />
        {clicked && <p>클릭한 색상: {clicked}</p>}
      </Wrapper>
    );
}

export default Blocks;
```

styled-components

- **Blocks 모형**

```
const Wrapper = styled.div`
  display: flex;
  flex-direction: row;
  align-items: flex-start;
  justify-content: center;
  gap: 20px;
  padding: 40px;
  background-color: lightgray;
  height: 100vh;
`;

const Block = styled.div`
  width: 100px;
  height: 100px;
  background-color: ${props => props.color};
  border-radius: 8px;
  cursor: pointer;
`;
```